

ASSIGNMENT - 11.3

Lab 11: Data Structures with AI Implementing Fundamental Data

Structures using AI Assistance

NAME: Y.sahasra

HTNO: 2303A51269

BTNO:19

Task 1: Smart Contact Manager (Arrays & Linked Lists)

Scenario

SR University's student club requires a simple Contact Manager Application to store members' names and phone numbers. The system should support efficient addition, searching, and deletion of contacts.

Tasks

1. Implement the contact manager using arrays (lists).
2. Implement the same functionality using a linked list for dynamic memory allocation.
3. Implement the following operations in both approaches:
 - o Add a contact
 - o Search for a contact
 - o Delete a contact
4. Use GitHub Copilot to assist in generating search and delete methods.
5. Compare array vs. linked list approaches with respect to:
 - o Insertion efficiency
 - o Deletion efficiency

Expected Outcome

- Two working implementations (array-based and linked-list-based).
- A brief comparison explaining performance differences.

CODE:

This screenshot shows the VS Code editor with a Python file named `11.3 Task 1.py`. The code implements an `Array Based Contact Manager` using a `list` to store contact information. The `CONTACT_MANAGER_ARRAY.py` class includes methods for adding, searching, deleting, and displaying contacts.

```
1  # -----
2  # Array Based Contact Manager
3  # -----
4
5  class ContactManagerArray:
6
7      def __init__(self):
8          self.contacts = [] # List to store contacts
9
10     def add_contact(self, name, phone):
11         """Add a new contact"""
12         self.contacts.append({"name": name, "phone": phone})
13         print("Contact added successfully!")
14
15     def search_contact(self, name):
16         """Search contact by name"""
17         for contact in self.contacts:
18             if contact["name"] == name:
19                 return contact
20         return None
21
22     def delete_contact(self, name):
23         """Delete contact by name"""
24         for i in range(len(self.contacts)):
25             if self.contacts[i]["name"] == name:
26                 del self.contacts[i]
27                 print("Contact deleted successfully!")
28                 return True
29         return False
30
31     def display_contacts(self):
32         """Display all contacts"""
33         if not self.contacts:
34             print("No contacts found.")
35             return
36         for contact in self.contacts:
```

This screenshot shows the VS Code editor with a Python file named `11.3 Task 1.py`. The code implements a `Linked List Contact Manager` using a `Node` class and a `head` pointer. The `CONTACT_MANAGER_LINKED_LIST.py` class includes methods for adding, searching, and displaying contacts.

```
5  class ContactManagerArray:
31     def display_contacts(self):
38         print(contact["name"], "-", contact["phone"])
39
40     # -----
41     # Linked List Contact Manager
42     # -----
43
44     class Node:
45
46         def __init__(self, name, phone):
47             self.name = name
48             self.phone = phone
49             self.next = None
50
51     class ContactManagerLinkedList:
52
53         def __init__(self):
54             self.head = None
55
56         def add_contact(self, name, phone):
57             """Add contact at beginning"""
58             new_node = Node(name, phone)
59             new_node.next = self.head
60             self.head = new_node
61             print("Contact added successfully!")
62
63         def search_contact(self, name):
64             """Search contact by name"""
65             current = self.head
66             while current:
67                 if current.name == name:
68                     return {"name": current.name, "phone": current.phone}
69                 current = current.next
70
71         def display_contacts(self):
72             """Display all contacts"""
```

```
11.3 Task 1.py
class ContactManagerLinkedList:
    def search_contact(self, name):
        return None

    def delete_contact(self, name):
        """Delete contact by name"""
        current = self.head
        prev = None

        while current:
            if current.name == name:
                # If first node
                if prev is None:
                    self.head = current.next
                else:
                    prev.next = current.next

                print("Contact deleted successfully!")
                return True

            prev = current
            current = current.next

        return False

    def display_contacts(self):
        """Display all contacts"""
        if self.head is None:
            print("No contacts found.")
            return

        current = self.head

        while current:
            print(current.name, "...", current.phone)
```

```
11.3 Task 1.py
class ContactManagerLinkedList:
    def display_contacts(self):
        current = current.next

        # -----
        # Main Program (Testing)
        # -----
        if __name__ == "__main__":
            print("=====")
            print(" SMART CONTACT MANAGER SYSTEM")
            print("=====")

            # ----- Array Version -----
            print("\n--- Array Contact Manager ----")

            array_manager = ContactManagerArray()

            array_manager.add_contact("sara", "9876543210")
            array_manager.add_contact("deepu", "9123456789")
            array_manager.add_contact("nishi", "9988776655")

            print("\nSearching for sara:")
            print(array_manager.search_contact("sara"))

            print("\nAll Contacts:")
            array_manager.display_contacts()

            print("\nDeleting deepu:")
            array_manager.delete_contact("deepu")

            print("\nContacts After Deletion:")
            array_manager.display_contacts()
```

OUTPUT:

```
> & C:\Users\yvasa\AppData\Local\Programs\Python\Python311\python.exe "c:/Users/yvasa/Downloads/11.3 Task 1.py"
=====
SMART CONTACT MANAGER SYSTEM
=====

---- Array Contact Manager ----
Contact added successfully!
Contact added successfully!
Contact added successfully!

Searching for sara:
{'name': 'sara', 'phone': '9876543210'}

All Contacts:
sara - 9876543210
deepu - 9123456789
nishi - 9988776655

Deleting deepu:
Contact deleted successfully!

Contacts After Deletion:
sara - 9876543210
nishi - 9988776655

---- Linked List Contact Manager ----
Contact added successfully!
Contact added successfully!
Contact added successfully!

Searching for sara:
{'name': 'sara', 'phone': '9876543210'}

All Contacts:
nishi - 9988776655
deepu - 9123456789
sara - 9876543210

Deleting deepu:
```

Task 2: Library Book Search System (Queues & Priority Queues)

Scenario

The SRU Library manages book borrow requests. Students and faculty submit requests, but faculty requests must be prioritized over student requests.

Tasks

1. Implement a Queue (FIFO) to manage book requests.
2. Extend the system to a Priority Queue, prioritizing faculty requests.
3. Use GitHub Copilot to assist in generating:

o enqueue() method o

dequeue() method

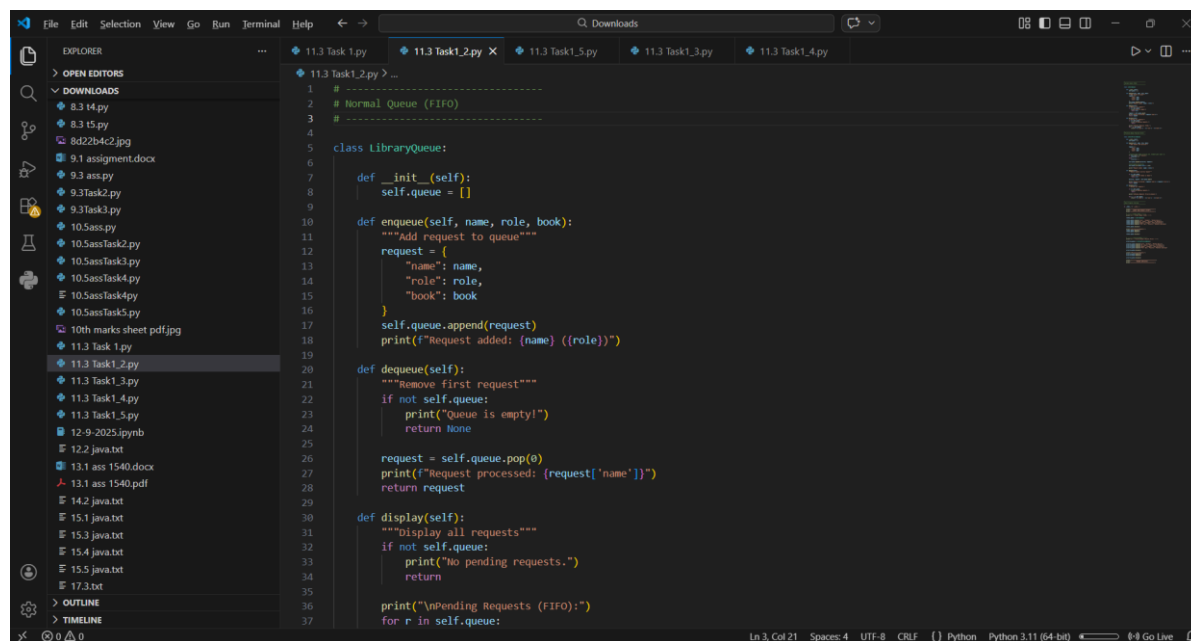
4. Test the system with a mix of student and faculty requests.

Expected Outcome

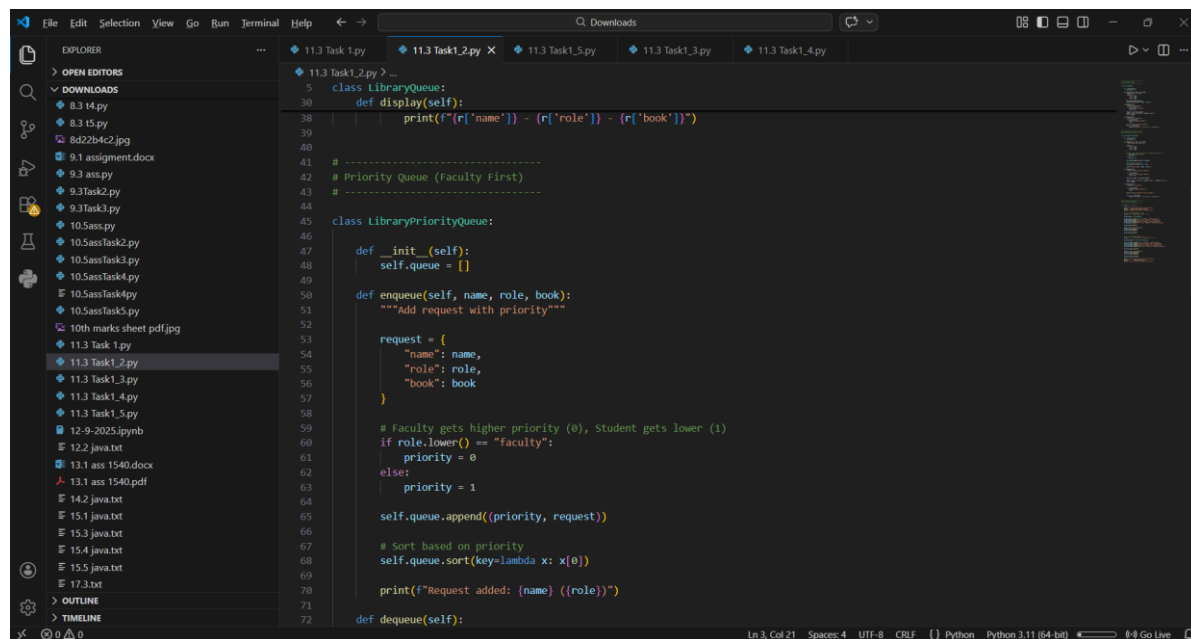
Working queue and priority queue implementations.

- Correct prioritization of faculty requests.

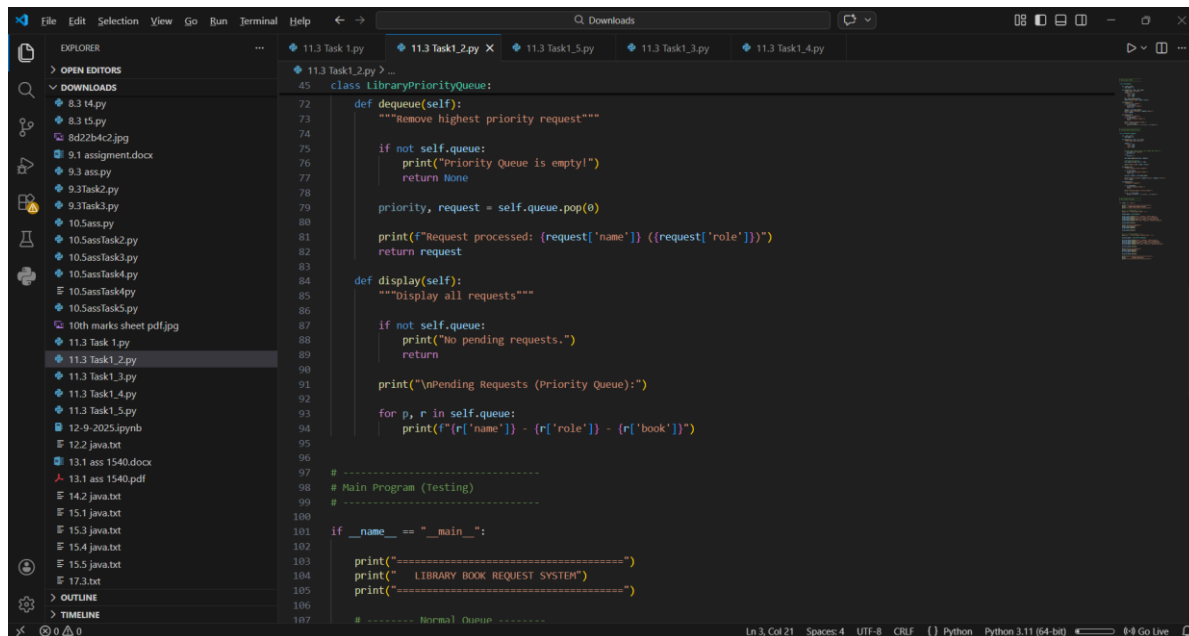
CODE:



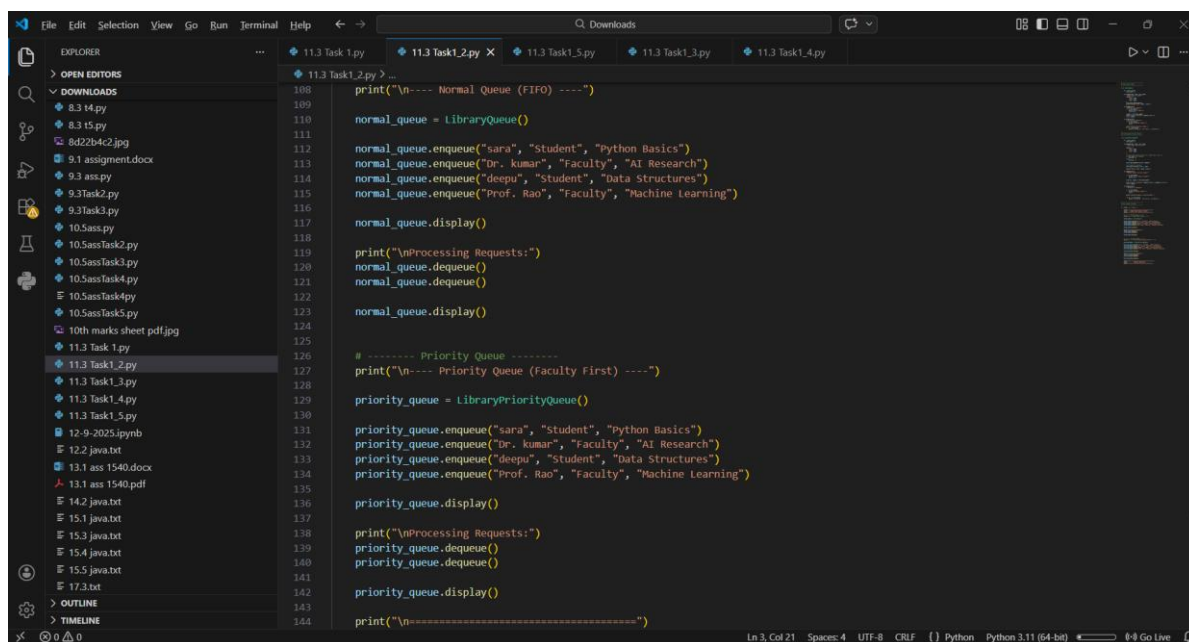
```
1 # -----
2 # Normal Queue (FIFO)
3 # -----
4
5 class LibraryQueue:
6
7     def __init__(self):
8         self.queue = []
9
10    def enqueue(self, name, role, book):
11        """Add request to queue"""
12        request = {
13            "name": name,
14            "role": role,
15            "book": book
16        }
17        self.queue.append(request)
18        print(f"Request added: {name} ({role})")
19
20    def dequeue(self):
21        """Remove first request"""
22        if not self.queue:
23            print("Queue is empty!")
24            return None
25
26        request = self.queue.pop(0)
27        print(f"Request processed: {request['name']}")
28        return request
29
30    def display(self):
31        """Display all requests"""
32        if not self.queue:
33            print("No pending requests.")
34            return
35
36        print("\nPending Requests (FIFO):")
37        for r in self.queue:
```



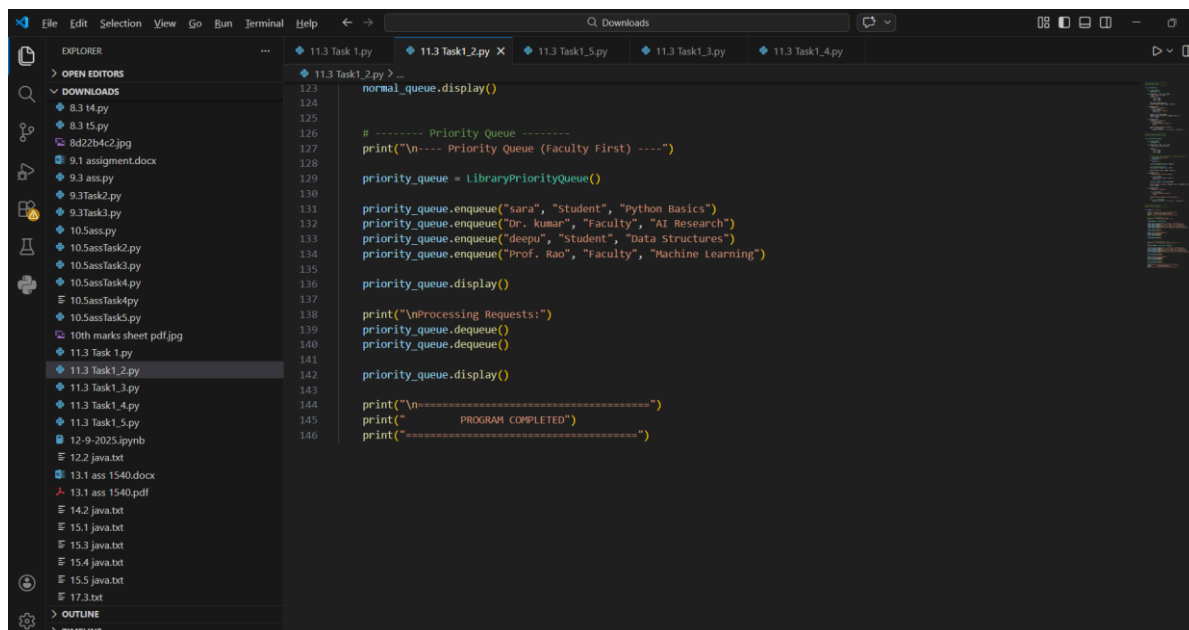
```
5 class LibraryQueue:
6     def display(self):
7         print(f"{r['name']} - {r['role']} - {r['book']}")
8
9 # -----
10 # Priority Queue (Faculty First)
11 # -----
12
13 class LibraryPriorityQueue:
14
15     def __init__(self):
16         self.queue = []
17
18     def enqueue(self, name, role, book):
19        """Add request with priority"""
20
21        request = {
22            "name": name,
23            "role": role,
24            "book": book
25        }
26
27        # Faculty gets higher priority (0), Student gets lower (1)
28        if role.lower() == "faculty":
29            priority = 0
30        else:
31            priority = 1
32
33        self.queue.append((priority, request))
34
35        # Sort based on priority
36        self.queue.sort(key=lambda x: x[0])
37
38        print(f"Request added: {name} ({role})")
39
40    def dequeue(self):
```



```
113 Task1_2.py > ...
45 class LibraryPriorityQueue:
72     def dequeue(self):
73         """Remove highest priority request"""
74
75         if not self.queue:
76             print("Priority Queue is empty!")
77             return None
78
79         priority, request = self.queue.pop(0)
80
81         print(f"Request processed: {request['name']} ({request['role']})")
82         return request
83
84     def display(self):
85         """Display all requests"""
86
87         if not self.queue:
88             print("No pending requests.")
89             return
90
91         print("\nPending Requests (Priority Queue):")
92
93         for p, r in self.queue:
94             print(f"[{r['name']}] - [{r['role']}] - [{r['book']}]")
95
96
97 # -----
98 # Main Program (Testing)
99 # -----
100
101 if __name__ == "__main__":
102
103     print("=====")
104     print("  LIBRARY BOOK REQUEST SYSTEM")
105     print("=====")
106
107 # ----- Normal Queue -----
```

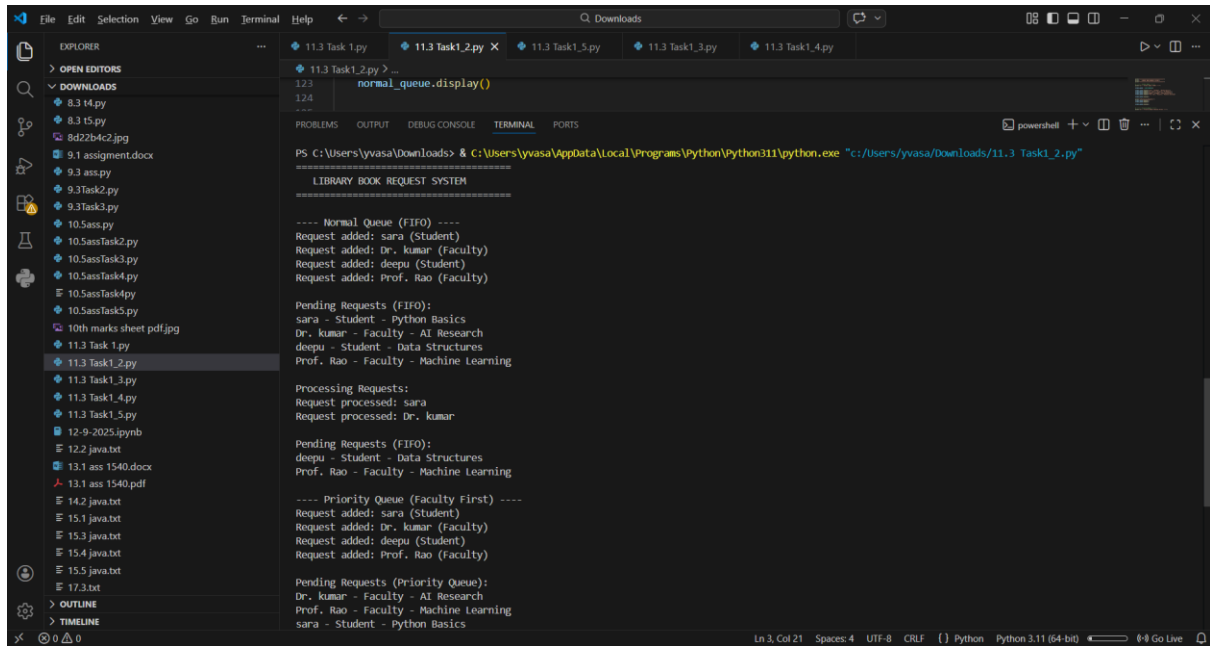


```
108 print("\n---- Normal Queue (FIFO) ----")
109
110 normal_queue = LibraryQueue()
111
112 normal_queue.enqueue("sara", "Student", "Python Basics")
113 normal_queue.enqueue("Dr. kumar", "Faculty", "AI Research")
114 normal_queue.enqueue("deepu", "Student", "Data Structures")
115 normal_queue.enqueue("Prof. Rao", "Faculty", "Machine Learning")
116
117 normal_queue.display()
118
119 print("\nProcessing Requests:")
120 normal_queue.dequeue()
121 normal_queue.dequeue()
122
123 normal_queue.display()
124
125 # ----- Priority Queue -----
126 print("\n---- Priority Queue (Faculty First) ----")
127
128 priority_queue = LibraryPriorityQueue()
129
130 priority_queue.enqueue("sara", "Student", "Python Basics")
131 priority_queue.enqueue("Dr. kumar", "Faculty", "AI Research")
132 priority_queue.enqueue("deepu", "Student", "Data Structures")
133 priority_queue.enqueue("Prof. Rao", "Faculty", "Machine Learning")
134
135 priority_queue.display()
136
137 print("\nProcessing Requests:")
138 priority_queue.dequeue()
139 priority_queue.dequeue()
140
141 priority_queue.display()
142
143 print("\n=====")
144
```



```
123 normal_queue.display()
124
125 # ----- Priority Queue -----
126 print("\n---- Priority Queue (Faculty First) ----")
127
128 priority_queue = LibraryPriorityQueue()
129
130 priority_queue.enqueue("sara", "Student", "Python Basics")
131 priority_queue.enqueue("Dr. kumar", "Faculty", "AI Research")
132 priority_queue.enqueue("deepu", "Student", "Data Structures")
133 priority_queue.enqueue("Prof. Rao", "Faculty", "Machine Learning")
134
135 priority_queue.display()
136
137 print("\nProcessing Requests:")
138 priority_queue.dequeue()
139 priority_queue.dequeue()
140
141 priority_queue.display()
142
143 print("\n=====")
144 print("  PROGRAM COMPLETED")
145 print("=====")
146
```

OUTPUT:



The screenshot shows a VS Code editor with a file explorer on the left and a terminal window at the bottom. The file explorer shows a list of files in the 'Downloads' folder, including '11.3 Task 1.py', '11.3 Task 1.2.py', '11.3 Task 1.3.py', '11.3 Task 1.4.py', '11.3 Task 1.5.py', '12-9-2025.ipynb', '12.2 java.txt', '13.1 ass 1540.docx', '13.1 ass 1540.pdf', '14.2 java.txt', '15.1 java.txt', '15.3 java.txt', '15.4 java.txt', '15.5 java.txt', and '17.3.txt'. The terminal window shows the output of the script '11.3 Task 1.2.py', which is a library book request system. The output is as follows:

```
PS C:\Users\yvasa\Downloads> & C:\Users\yvasa\AppData\Local\Programs\Python\Python311\python.exe "c:/Users/yvasa/Downloads/11.3 Task1_2.py"

LIBRARY BOOK REQUEST SYSTEM
=====

---- Normal Queue (FIFO) ----
Request added: sara (Student)
Request added: Dr. kumar (Faculty)
Request added: deepu (Student)
Request added: Prof. Rao (Faculty)

Pending Requests (FIFO):
sara - Student - Python Basics
Dr. kumar - Faculty - AI Research
deepu - Student - Data Structures
Prof. Rao - Faculty - Machine Learning

Processing Requests:
Request processed: sara
Request processed: Dr. kumar

Pending Requests (FIFO):
deepu - Student - Data Structures
Prof. Rao - Faculty - Machine Learning

---- Priority Queue (Faculty First) ----
Request added: sara (Student)
Request added: Dr. kumar (Faculty)
Request added: deepu (Student)
Request added: Prof. Rao (Faculty)

Pending Requests (Priority Queue):
Dr. kumar - Faculty - AI Research
Prof. Rao - Faculty - Machine Learning
sara - Student - Python Basics
```

Task 3: Emergency Help Desk (Stack Implementation)

Scenario

SR University's IT Help Desk receives technical support tickets from students and staff.

While tickets are received sequentially, issue escalation follows a

Last-In, First-Out (LIFO) approach.

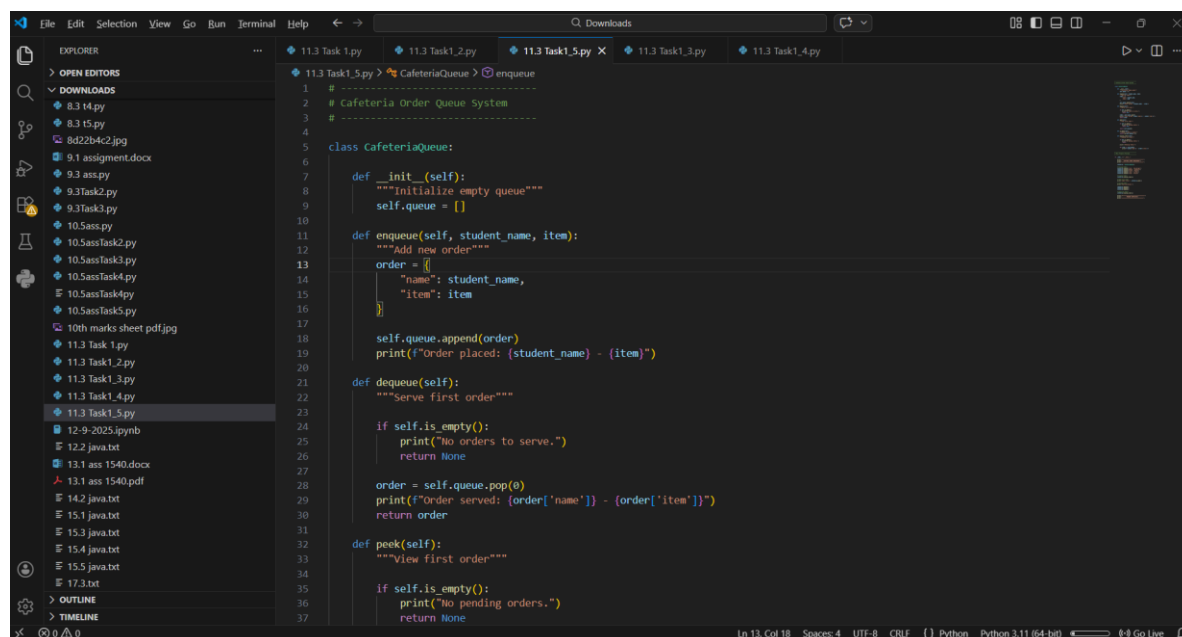
Tasks

1. Implement a Stack to manage support tickets.
2. Provide the following operations:
 - o push(ticket)
 - o pop()
 - o peek()
3. Simulate at least five tickets being raised and resolved.
4. Use GitHub Copilot to suggest additional stack operations such as:
 - o Checking whether the stack is empty
 - o Checking whether the stack is full (if applicable)

Expected Outcome

- Functional stack-based ticket management system.
- Clear demonstration of LIFO behavior.

CODE:



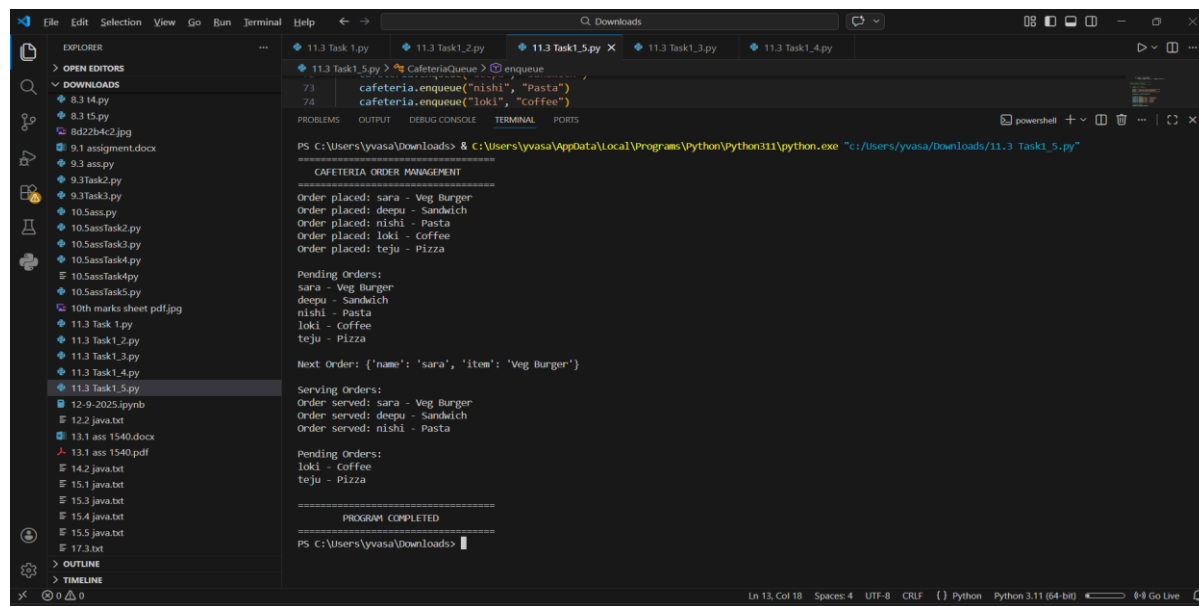
```
1  # -----
2  # Cafeteria Order Queue System
3  # -----
4
5  class CafeteriaQueue:
6
7      def __init__(self):
8          """Initialize empty queue"""
9          self.queue = []
10
11      def enqueue(self, student_name, item):
12          """Add new order"""
13          order = {
14              "name": student_name,
15              "item": item
16          }
17          self.queue.append(order)
18          print(f"Order placed: {student_name} - {item}")
19
20      def dequeue(self):
21          """Serve first order"""
22
23          if self.is_empty():
24              print("No orders to serve.")
25              return None
26
27          order = self.queue.pop(0)
28          print(f"Order served: {order['name']} - {order['item']}")
29          return order
30
31      def peek(self):
32          """View first order"""
33
34          if self.is_empty():
35              print("No pending orders.")
36              return None
37
38      def is_empty(self):
39          """Check if queue is empty"""
40          return len(self.queue) == 0
41
42      def is_full(self):
43          """Check if queue is full (not applicable for this implementation)"""
44          return False
```



```
File Edit Selection View Go Run Terminal Help
11.3 Task1.py 11.3 Task1_2.py 11.3 Task1_5.py X 11.3 Task1_3.py 11.3 Task1_4.py
11.3 Task1_5.py > CafeteriaQueue > enqueue
5 class CafeteriaQueue:
32     def peek(self):
38         return self.queue[0]
48
41     def is_empty(self):
42         """Check if queue is empty"""
43         return len(self.queue) == 0
44
45     def display_orders(self):
46         """Display all orders"""
47
48         if self.is_empty():
49             print("No pending orders.")
50             return
51
52         print("\nPending Orders:")
53
54         for order in self.queue:
55             print(f'{order["name"]} - {order["item"]}')
56
57
58 # -----
59 # Main Program (Testing)
60 # -----
61
62 if __name__ == "__main__":
63
64     print("=====")
65     print("  CAFETERIA ORDER MANAGEMENT")
66     print("=====")
67
68     cafeteria = CafeteriaQueue()
69
70 # Placing Orders
71 cafeteria.enqueue("sara", "Veg Burger")
72 cafeteria.enqueue("deepu", "Sandwich")
```

```
73 cafeteria.enqueue("nishi", "Pasta")
74 cafeteria.enqueue("loki", "Coffee")
75 cafeteria.enqueue("teju", "Pizza")
76
77 # Display Orders
78 cafeteria.display_orders()
79
80 # Peek First Order
81 print("\nNext Order:", cafeteria.peek())
82
83 # Serving Orders
84 print("\nServing Orders:")
85
86 cafeteria.dequeue()
87 cafeteria.dequeue()
88 cafeteria.dequeue()
89
90 # Remaining Orders
91 cafeteria.display_orders()
92
93 print("\n=====")
94 print("  PROGRAM COMPLETED")
95 print("=====")
```

OUTPUT:



```
11.3 Task1_5.py > CafeteriaQueue > enqueue
73 cafeteria.enqueue("nishi", "Pasta")
74 cafeteria.enqueue("loki", "Coffee")

PS C:\Users\yvasa\Downloads> & C:\Users\yvasa\AppData\Local\Programs\Python\Python311\python.exe "c:\Users\yvasa\Downloads\11.3 Task1_5.py"

CAFETERIA ORDER MANAGEMENT
=====
Order placed: sara - Veg Burger
Order placed: deepu - Sandwich
Order placed: nishi - Pasta
Order placed: loki - Coffee
Order placed: teju - Pizza

Pending Orders:
sara - Veg Burger
deepu - Sandwich
nishi - Pasta
loki - Coffee
teju - Pizza

Next Order: {'name': 'sara', 'item': 'Veg Burger'}

Serving Orders:
Order served: sara - Veg Burger
Order served: deepu - Sandwich
Order served: nishi - Pasta

Pending Orders:
loki - Coffee
teju - Pizza

=====
PROGRAM COMPLETED
=====

PS C:\Users\yvasa\Downloads>
```

Task 4: Hash Table

Objective

To implement a Hash Table and understand collision handling.

Task Description

Use AI to generate a hash table with:

- Insert
 - Search
 - Delete
- Starter Code class HashTable:

pass

Expected Outcome

- Collision handling using chaining
- Well-commented methods

CODE:

This screenshot shows the first part of a Python script in VS Code. The Explorer panel on the left lists various files, with '11.3 Task1_4.py' selected. The main editor displays the following code:

```
1 # -----
2 # Hash Table Implementation
3 # Using Chaining for Collision Handling
4 # -----
5
6 class HashTable:
7
8     def __init__(self, size=10):
9         """
10         Initialize hash table with given size
11         Each index contains a list (chain)
12         """
13         self.size = size
14         self.table = [[] for _ in range(size)]
15
16     def hash_function(self, key):
17         """
18         Generate hash value for a key
19         Converts key to number and maps to table size
20         """
21         return hash(key) % self.size
22
23     def insert(self, key, value):
24         """
25         Insert key-value pair into hash table
26         Handles collision using chaining
27         """
28
29         index = self.hash_function(key)
30
31         # Check if key already exists -> Update value
32         for pair in self.table[index]:
33             if pair[0] == key:
34                 pair[1] = value
35                 print(f"Updated: {key} -> {value}")
36                 return
```

This screenshot shows the second part of the Python script in VS Code. The Explorer panel on the left is the same, with '11.3 Task1_4.py' selected. The main editor displays the following code:

```
23 class HashTable:
24     def insert(self, key, value):
25         """
26         # If key not found -> Insert new pair
27         self.table[index].append((key, value))
28         print(f"Inserted: {key} -> {value}")
29
30     def search(self, key):
31         """
32         Search for a key in hash table
33         Returns value if found, else None
34         """
35
36         index = self.hash_function(key)
37
38         for pair in self.table[index]:
39             if pair[0] == key:
40                 return pair[1]
41
42         return None
43
44     def delete(self, key):
45         """
46         Delete key-value pair from hash table
47         """
48
49         index = self.hash_function(key)
50
51         for i, pair in enumerate(self.table[index]):
52             if pair[0] == key:
53                 self.table[index].pop(i)
54                 print(f"Deleted: {key}")
55                 return True
56
57         print(f"{key} not found")
58         return False
```

This screenshot shows the third part of the Python script in VS Code. The Explorer panel on the left is the same, with '11.3 Task1_4.py' selected. The main editor displays the following code:

```
6 class HashTable:
7     def display(self):
8         """
9         Display full hash table
10         """
11
12         print("\nHash Table:")
13
14         for i in range(self.size):
15             print(f"Index {i}: ", end=" ")
16
17             if not self.table[i]:
18                 print("Empty")
19             else:
20                 for pair in self.table[i]:
21                     print(f"{pair[0]}:{pair[1]}, ", end=" ")
22                 print("None")
23
24 # -----
25 # Main Program (Testing)
26 # -----
27
28 if __name__ == "__main__":
29
30     print("=====")
31     print("      HASH TABLE SYSTEM")
32     print("=====")
33
34     ht = HashTable(size=7)
35
36     # Insert values
37     ht.insert("Roll101", "sara")
38     ht.insert("Roll102", "depu")
39     ht.insert("Roll103", "nishi")
40     ht.insert("Roll110", "loki")
41     ht.insert("Roll117", "seju") # collision possible
```

```
102
103 # Insert values
104 ht.insert("Roll101", "sara")
105 ht.insert("Roll102", "deepu")
106 ht.insert("Roll103", "nishi")
107 ht.insert("Roll110", "loki")
108 ht.insert("Roll117", "teju") # Collision possible
109
110 # Display Table
111 ht.display()
112
113 # Search
114 print("\nSearching Roll102:", ht.search("Roll102"))
115 print("Searching Roll200:", ht.search("Roll200"))
116
117 # Delete
118 ht.delete("Roll103")
119 ht.delete("Roll999") # Not present
120
121 # Display after deletion
122 ht.display()
123
124 print("\n=====")
125 print("          PROGRAM COMPLETED")
126 print("=====")
```

OUTPUT:

```
=====
PROGRAM COMPLETED
PS C:\Users\yvasa\Downloads> & C:\Users\yvasa\AppData\Local\Programs\Python\Python311\python.exe "c:/Users/yvasa/Downloads/11.3 Task1_4.py"
=====
HASH TABLE SYSTEM
=====
Inserted: Roll101 -> sara
Inserted: Roll102 -> deepu
Inserted: Roll103 -> nishi
Inserted: Roll110 -> loki
Inserted: Roll117 -> teju

Hash Table:
Index 0: Roll110:loki -> None
Index 1: Roll101:sara -> None
Index 2: Roll102:deepu -> Roll117:teju -> None
Index 3: Empty
Index 4: Empty
Index 5: Empty
Index 6: Roll103:nishi -> None

Searching Roll102: deepu
Searching Roll200: None
Deleted: Roll103
Roll999 not found

Hash Table:
Index 0: Roll110:loki -> None
Index 1: Roll101:sara -> None
Index 2: Roll102:deepu -> Roll117:teju -> None
Index 3: Empty
Index 4: Empty
Index 5: Empty
Index 6: Empty

=====
PROGRAM COMPLETED
=====
PS C:\Users\yvasa\Downloads>
```

Task 5: Real-Time Application Challenge

Scenario

Design a Campus Resource Management System with the following features:

- Student Attendance Tracking
- Event Registration System
- Library Book Borrowing
- Bus Scheduling System
- Cafeteria Order Queue

Student Tasks

1. Choose the most appropriate data structure for each feature.
2. Justify your choice in 2–3 sentences.
3. Implement one selected feature using AI-assisted code generation.

Expected Outcome

- Mapping table: Feature → Data Structure → Justification
- One fully working Python implementation

Note: Report should be submitted as a word document for all tasks in a single document with prompts, comments & code explanation, and output and if required, screenshots.

CODE:

This screenshot shows the first part of a Python script in VS Code. The Explorer panel on the left lists various files in the 'Downloads' folder, with '11.3 Task1_5.py' selected. The main editor displays the following code:

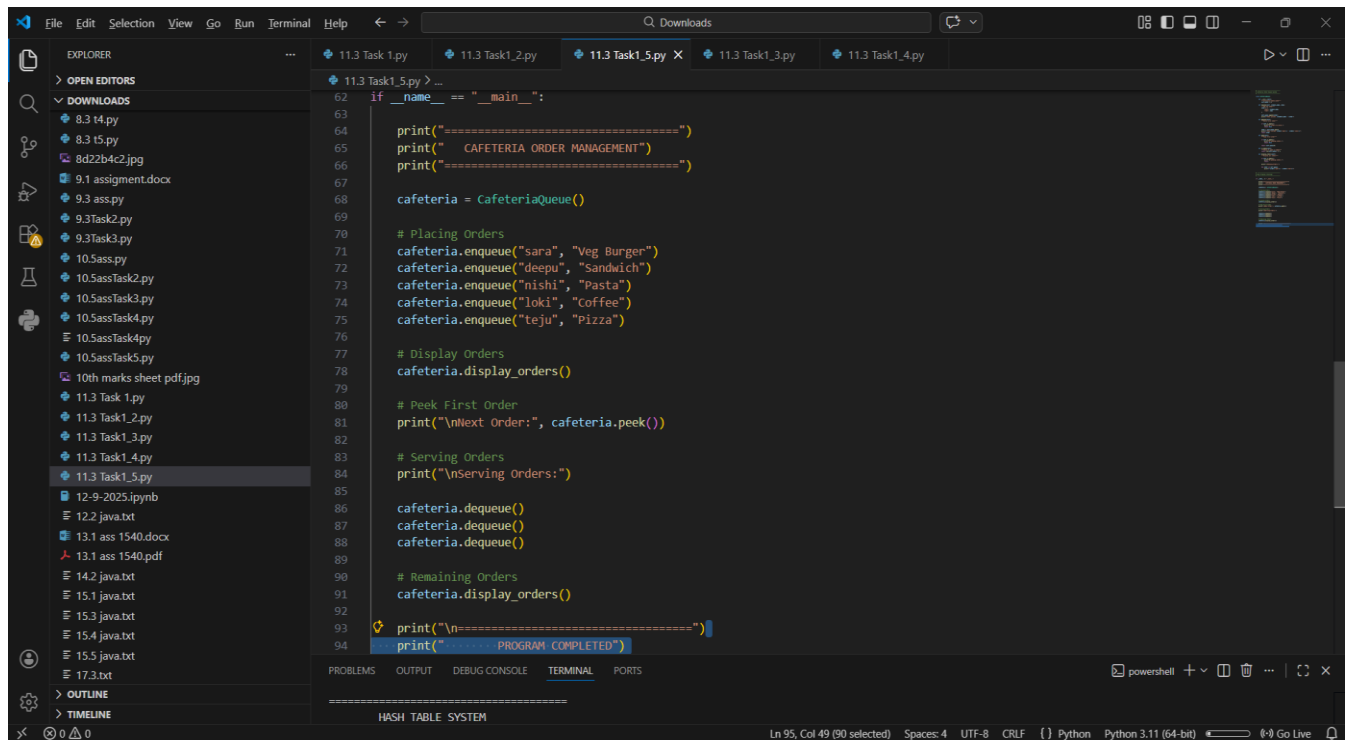
```
1 # -----
2 # Cafeteria Order Queue System
3 # -----
4
5 class CafeteriaQueue:
6
7     def __init__(self):
8         """Initialize empty queue"""
9         self.queue = []
10
11     def enqueue(self, student_name, item):
12         """Add new order"""
13         order = {
14             "name": student_name,
15             "item": item
16         }
17
18         self.queue.append(order)
19         print(f"Order placed: {student_name} - {item}")
20
21     def dequeue(self):
22         """Serve first order"""
23
24         if self.is_empty():
25             print("No orders to serve.")
26             return None
27
28         order = self.queue.pop(0)
29         print(f"Order served: {order['name']} - {order['item']}")
30         return order
31
32     def peek(self):
33         """View first order"""
```

The bottom status bar indicates the file is at line 95, column 49, with 90 characters selected. The editor is using Python 3.11 (64-bit).

This screenshot shows the second part of the Python script in VS Code, continuing from the previous view. The code includes the following methods and a main testing block:

```
32     def peek(self):
33         if self.is_empty():
34             print("No pending orders.")
35             return None
36
37         return self.queue[0]
38
39     def is_empty(self):
40         """check if queue is empty"""
41         return len(self.queue) == 0
42
43     def display_orders(self):
44         """Display all orders"""
45
46         if self.is_empty():
47             print("No pending orders.")
48             return
49
50         print("\nPending Orders:")
51
52         for order in self.queue:
53             print(f"{order['name']} - {order['item']}")
54
55 # -----
56 # Main Program (Testing)
57 # -----
58
59 if __name__ == "__main__":
60     print("=====")
```

The bottom status bar shows the cursor is now at line 95, column 49, with 90 characters selected. The editor remains in Python 3.11 (64-bit).



OUTPUT:

```
08 | Cafeteria = CafeteriaQueue()  
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS powershell +  
  
PROGRAM COMPLETED  
=====
```

```
PS C:\Users\yvasa\Downloads> & C:\Users\yvasa\AppData\Local\Programs\Python\Python311\python.exe "c:/Users/yvasa/Downloads/11.3 Task1_5.py"  
=====
```

```
CAFETERIA ORDER MANAGEMENT  
=====
```

```
Order placed: sara - Veg Burger  
Order placed: deepu - Sandwich  
Order placed: nishi - Pasta  
Order placed: loki - Coffee  
Order placed: teju - Pizza
```

```
Pending Orders:  
sara - Veg Burger  
deepu - Sandwich  
nishi - Pasta  
loki - Coffee  
teju - Pizza
```

```
Next Order: {'name': 'sara', 'item': 'Veg Burger'}
```

```
Serving Orders:  
Order served: sara - Veg Burger  
Order served: deepu - Sandwich  
Order served: nishi - Pasta
```

```
Pending Orders:  
loki - Coffee  
teju - Pizza
```

```
=====
```

```
PROGRAM COMPLETED  
=====
```

```
PS C:\Users\yvasa\Downloads> |
```


OUTPUT:

